

Names, NetIDs, emails:

Jackson LeJeune, jleje2, jleje2@uic.edu

Dean Tubongbanua, dtubon2, dtubon2@uic.edu

Alec Sanders, asand41, asand41@uic.edu

Bryan Dominguez, bdomi2, bdomi2@uic.edu

Group Number: 22**Project Name:** Arduino Telegraph

Project Abstract: Using Arduino boards, LCD screens, LoRa radio transceivers, and a few buttons, we will create a system that, given input of dots and dashes, will send an encoded message to other Arduino boards, using radio signals, that will decode the message and display it on an LCD screen. Users will select specific “channels” to broadcast on using a joystick, and any other Arduinos on that channel will receive any message broadcasted over it.

Detailed Project Ideas:**1. Overall Description of Project Idea**

Inspired by wireless radio telegraphs, we wanted to create a Morse code communication device that is capable of both transmitting and receiving messages. The message will be represented by various outputs (light, sound, and text). The transmission and decoding of signals will be done between two LoRa modules, over specific radio frequencies selectable by user. The result is a unique blend of analog and digital communication. If a user receives more than one message, it will be stored in a queue and can be viewed later.

2. Final Project Design stating how Multiple Arduinos will be used

Each Arduino board will act as its own receiver and transmitter: think of them as their own telegraph station. Each board will accept user input to create and transmit a message encoded in Morse code on a selected “channel” (radio frequency). Any boards tuned to that frequency will receive the message, decode it, and display it on their LCD screen. With the message queue, multiple people can send messages without overwriting the other.

3. Final Plan for Use and Communication between the multiple Arduinos

The overall plan for use and communication between our multiple Arduino boards involves RF transceivers as the primary function for transmitting and receiving messages, since they are cost-effective and they provide a reliable connection which gives us less worry. The messages will go through a few layers of encoding/decoding on their way from button input on one Arduino to a displayed message on the other Arduino. 1. The sender inputs their message as a series of dots, dashes, and stops, and then presses the send button. 2. The message is converted to Serial using SoftwareSerial, so that any Arduino board can read it. 3. The message is encoded with Huffman binary encoding and sent through the LoRa. 4. The message is received by the

other LoRa(s), and the process is done in reverse, and the message is displayed on the LCD of the receiver once the receiver desires to read it.

4. Final Project Design stating Expected Inputs/Outputs

The device(s) waits for user input (either message buttons or channel joystick) while also listening for any messages sent over the radio channel. The joystick enables “scrolling” through the predefined channels, when pressed, the joystick sends a channel change command to the LoRa. Upon confirmation of active channel, the Arduino updates 7-segment display to the current channel. The DOT (‘.’), DASH (‘-’), SPACE (‘ ’), and CLEAR buttons will modify the message buffer, and the SEND button will transmit the message currently in the buffer to any devices listening on the channel. As dots and dashes are entered, an LED will flash and a speaker will emit sound in sync with the buttons. Once a complete Morse character is typed (indicated by a single space) the character is decoded and displayed on the LCD. If the Morse code is invalid, a placeholder ‘#’ will be displayed. While the user is composing a message, the decimal point on the 7-segment display will be lit if they have a message waiting, a sort of “You’ve Got Mail” indicator. Once they return to the ready state and press the send button, the message will be opened and displayed until another button is pressed. The joystick being moved left and right will cycle the radio channels up and down between 0 and 9, in steps of 0.2 mHz.

5. Final description of the original work being attempted by your project.

The examples of Arduino telegraphs that we researched are quite basic and build the transmitter and receiver separately. We combined these features and implemented multi-board communication by broadcasting to a chosen channel. Users select the specific radio frequency channels, restricted to ones over which we will allow communication. The LCD makes composing messages easier, allowing one to preview and edit their message. The system will be universal across Arduino boards, as the only component required to be similar is the radio chip. Additionally, the message queue has not been implemented elsewhere from what we have seen.

6. How to Build an Arduino Telegraph

- a. Wiring the LCD: using 4 female-to-male wires, wire GND->breadboard ground, VCC->breadboard 5V power, SDA->Arduino A4, SCL->Arduino A5
- b. Wiring the buttons: Place a button on the breadboard so that the pins bridge the gap in the center. On one side of the breadboard, wire one pin of the button to 5V power, and then wire the other pin to ground, passing through a 10,000 Ω resistor. On the other side of the breadboard from the power and ground, wire the button pin on the same row of the resistor to Arduino D8.
Repeat the above steps four more times, wiring the output to Arduino pins D9, D11, D12, and D13. The buttons correspond to inputs as follows: 8=dot, 9=dash, 11=space, 12=clear, 13=send.

- c. Wiring the sound system: Place the speaker on the breadboard. Connect the positive end of the speaker to digital pin 3, and negative to ground.. Place the potentiometer on the breadboard, wire the middle (wiper) pin to analog A2, and the left and right pins to 5V and ground (it doesn't matter which ones for power and ground).
- d. Wiring the LED: place the LED on the breadboard, ideally near the buttons. Connect the longer pin of the LED to Arduino digital pin D2, and the short pin of the LED to ground through a 220Ω resistor.
- e. Wiring the joystick: Wire the GND pin to breadboard ground, and +5V pin to breadboard power. Wire VRx (horizontal) and VRy (vertical) to A0 and A1 respectively.
- f. Wiring the LoRa, Regulator, and Logic-Level Shifter:
 - i. Place the LoRa on the breadboard. Place the capacitor such that its pins are on the same row as the LoRa's 3.3V and ground pins. Wire those rows to breadboard power and ground.
 - ii. Near the LoRa, place the 3.3V regulator on one side of the breadboard. Place the logic-level shifter straddling the gap of the breadboard, with the low voltage side on the same half as the regulator. Connect the regulator's VIN pin to breadboard power, the ground pin to breadboard ground, and the output pin to the logic-level shifter's low voltage power.
 - iii. Wire the ground on both sides of the logic level shifter to breadboard ground, and the high voltage side to breadboard power. Wire LoRa's pin 0 to the LLS LV2 pin, and LoRa pin 1 to LLS LV1 pin. Wire HV1 and HV2 to Arduino digital pins D6 and D7
- g. Wire the Seven-segment display: Place the seven-segment on the breadboard straddling the breadboard's gap, and the 74HC595 shift register near it, with the writing on the register facing you, straddling the center of the breadboard. On the seven-segment, with the dot on the bottom, the top left pin is S0, and the top right is S4, and the bottom right is S9. On the register, the text oriented with the seven-segment, the top-left pin is pin R0, and the bottom-right pin is pin R15. R7 is the top-right pin and R8 the bottom-left one.
 - i. Wire register pins R0 and R6 to breadboard power, and pin R3 to ground. Wire pin R2 to D4, and pins R4 and R5 to D0 and D1, respectively.
 - ii. Wire R8 to S4, and R9 to S3. Wire R11 and R12 to S1 and S0, and then connect S2 to ground through a 220Ω resistor.
 - iii. Wire R10 to S5, and R1 to S6. Connect S7 to ground through a 220Ω resistor. Connect R13 to S8. Connect R14 to S9, and then connect R15 to ground.

7. How to Use the Arduino Telegraph

When starting your arduino telegraph, you will be in the “Ready” state, where the phrase “Ready for Input!” is displayed. There are 5 buttons to interact with, buttons 1-5. Button 1 is dot, button 2 is dash, 3 is space, 4 is delete, and 5 is send. If you press buttons 1-4, you will begin writing a message. If you press button 5, you will be taken to the message reading screen. To begin writing a message from the reading state, simply press any button 1-4.

All messages are composed of dots (1), dashes (2), and spaces (3). Each writable character (see table below) is composed of some combination of dots and dashes, and if the combination is invalid, a ‘#’ will be displayed instead. As you type dots and dashes, they will appear on the top line of the LCD, and the characters you have written will appear on the bottom. To advance to the next character, press button 3, space. If you want to add a space into your message, press space again after finishing a character. If you make a mistake, you can press button 4 (delete) to remove the last character. It will remove the character you are currently writing, and delete any others before it for more presses. Once you’re happy with your message, press button 5 to send your message.

You can select 10 different channels with the joystick. Pressing left and right on the joystick will tune up or down, although you cannot go below channel 0 or above channel 9. The channel number will be reflected in the seven-segment display. Messages both sent and received are exclusive to their channels, so if you send a message on channel 4, only other boards listening on channel 4 will hear it. On the seven-segment display, if the decimal point on the bottom right is lit, you have a message! To view it, return to the ready state by sending your current message, or clearing all your characters from your draft and pressing backspace once more. Then, from the ready state, press button 5 to view the message. If the light remains on after viewing the message, you have more than one! Press send to view the next one. To return to the drafting state from the reading state, press any button 1-4.

Letter	Morse
A	. -
B	- . . .
C	- . - .
D	- . .
E	.
F	. . - .
G	- - .
H	
I	. .
J	. - - -
K	- . -
L	. - . .
M	- -

Letter	Morse
N	- .
O	- - -
P	. - - .
Q	- - . -
R	. - .
S	. . .
T	-
U	. . -
V	. . . -
W	. - -
X	- . . -
Y	- . - -
Z	- - . .

Digit	Morse
0	- - - - -
1	. - - - -
2	. . - - -
3	. . . - -
4	 -
5	
6	-
7	- - . . .
8	- - - . .
9	- - - - .

Punctuation Mark	Morse
Error (also <HH>)	
& Ampersand	. - . . .
† Apostrophe	. - - - .
@ At sign	. - - . .
) Bracket, close (parenthesis)	- . - - -
(Bracket, open (parenthesis)	- . - . .
: Colon	- - - . . .
, Comma	- - . . -
= Equals sign	- . . . -
! Exclamation mark ⓘ	- . - . -
. Full-stop (period)	. - . - .
- Hyphen	-
* Multiplication sign (also x)	- . . -
% Percentage (literally: 0/0)	- - - - - . - . - - - -
+ Plus sign	. - . - .
" Quotation marks	. - . . .
? Question mark (query)	. . - - .
/ Slash	- . . - .

Timeline (past and future)

Week 1: 1/17 - 1/24

formation of group discord

preliminary organization

Initial brainstorming project ideas

Week 2: 1/24 - 2/1

Onboarded 4th teammate

Continued gathering ideas for the project

Completed milestone 1 (preliminary project idea) individually

Week 3: 2/1 – 2/19

Preparations for milestone 2 (basic project description):

- Committed to the wireless telegraph idea
- In-depth discussion regarding basic functionality
- Prepared document for milestone 2
- Submitted milestone 2

Week 4: 2/19 – 2/26

Researched different possible means of transmitting Morse code

- Wi-Fi
- Bluetooth

- LoRa
- RF on/off keying (OOK)

Purchased a pair of basic LoRa transceiver modules for experimentation.

Began design on I/O and the user interface as well as brainstorming for encoding/decoding methods.

Week 4: 2/26 – 3/5

Preparations for Milestone 3 (project commitment and contract)

- More refined description of features and functionality
- List of expected required materials
- First iteration timeline and weekly goals

Submitted document for milestone 3

Week 5: 3/5 – 3/12

Began planning for milestone 4 (Design presentation slides)

- Assigned slides for each team member to complete

Agreed on a date for the first virtual meeting

Division of labor during the design and build phase

- Alec and Dean: research/solve LoRa communication and write code for LoRa
- Jack: I/O code, encoding/decoding algorithm
- Bryan: Hardware design/diagrams for I/O, documentation

Week 6: 3/12 – 3/19

Big week with lots to do before milestone 5!

The first meeting virtual meeting held on 3/13

- Decided to commit to using RF instead of Bluetooth or Wi-Fi
 - o Would prefer OOK but open to other encoding methods
 - o Purchased two pairs of much more sophisticated LoRa modules
- Brought each other up to speed on what we had discovered and worked on up until this point

Submitted slides for milestone 4

Following the meeting, we did a significant amount of research and testing regarding the decision to use LoRa or OOK. The group decides the best solution is to send Huffman-encoded binary strings over LoRa.

Wrote the first functioning code to use improved LoRa modules to send strings between two Arduinos.

Completed first iteration of morse->string translation algorithm

Completed basic circuit design for user input, can display binary input as a decoded string on LCD.

Presented our project idea and progress for milestone 5, we peer reviewed another team's project for milestone 6.

Week 7: 3/19 – 3/26

Improved UI hardware/software design and morse->string translation.
functional now, lacking some features & output devices

Revised LoRa firmware to allow communication with Arduino R3

Week 8: 3/26 – 4/3

Preparing for milestone 7—writing this timeline.

- Jack – updated description and plan
- Bryan – updated diagram and code/pseudocode
- Dean – updated material and reference list
- Alec – document past progress and write a timeline for future goals.
 - o Assist others in completing their tasks

By the end of this week, we will:

- Be ready to submit milestone 7.
- Have a consolidated LoRa firmware capable of both sending and receiving
 - Have a functioning pair of Loras connected to I/O
 - Submit milestone 7

-----Post milestone 7-----

Week 9: 4/3 – 4/10

- Joystick and 7-segment display implemented, not integrated with LoRa
- Arduino and LoRa communication code written, untested.
- Group meeting to discuss current status and next steps.

Week 10: 4/10 – 4/17

- Arduino/LoRa communication code tested, confirmed
- 7-segment and Joystick integrated into broader code, implemented channel switching.
- Buzzer, LED, and potentiometer introduced, tested
- Huffman binary encoding begun, unfinished
- Communication between telegraphs confirmed
- Begun work on Milestone 9

Week 11: 4/17 - 4/24

- Board design finalized
- Buzzer implementation modified, finalized
- Two meetings to perform testing
- Documentation, code comments written
- Bugtesting and finalization of code
- Implemented “Message Waiting” system with 7-segment instead of second LED

Week 12: 4/24 – 5/2

- Completed project report
- Milestone 8, present device on Monday 4/28.
- Milestone 9, submit project report by 4/30.
- Milestone 10, submit teamwork assessment by 5/2.

Expected Materials (Per System):

- Wires
- 1x LED
- 1x LCD 16x2 Pin with i2c adapter
- 5x 10k Ω Resistors
- 3x 220 Ω Resistors
- 5x Pushbuttons
- 2x Full-Size Breadboards or 3x Half-Size breadboards
- 1x Arduino Uno R4 WiFi
- 1x Potentiometer
- 1x Speaker/Buzzer
- 1x 1-Digit 7-Segment Display
- 1x 74HC595 shift register
- LoRa SX1278 Long Range RF Wireless Power Module SX1276 5Km for Arduino
- AMS1117-3.3 DC-DC Step-Down Power Module AMS1117-3.3V Buck Module LDO 800MA MCIGICM 30 Values 1% Resistor Assortment, 600pcs 0 Ohm-1M Ohm 1/4W Metal Film Logic Level Shifter Converter Module Bidirectional 3.3V to 5V, 5V to 3.3V UART, IIC, SPI, 1-Wire 4 Channel
- Electrolytic in-line aluminum Capacitor Assortment Kit from 0.1uF to 1000uF 10V 16V 25V 50V
- 6 PCS 400 Pin Solderless Board Kit
- PCB Prototype Board, Large, Solderable

References:

<https://www.youtube.com/watch?v=r-jwNIMQPIk>

<https://github.com/sandeepmistry/arduino-LoRa>

<https://www.ecfr.gov/current/title-47/chapter-I/subchapter-A/part-15/subpart-C/subject-group-ECFR2f2e5828339709e/section-15.247>

<https://how2electronics.com/interfacing-sx1278-lora-module-with-arduino/>

<https://docs.arduino.cc/tutorials/uno-r4-wifi/wifi-examples/>

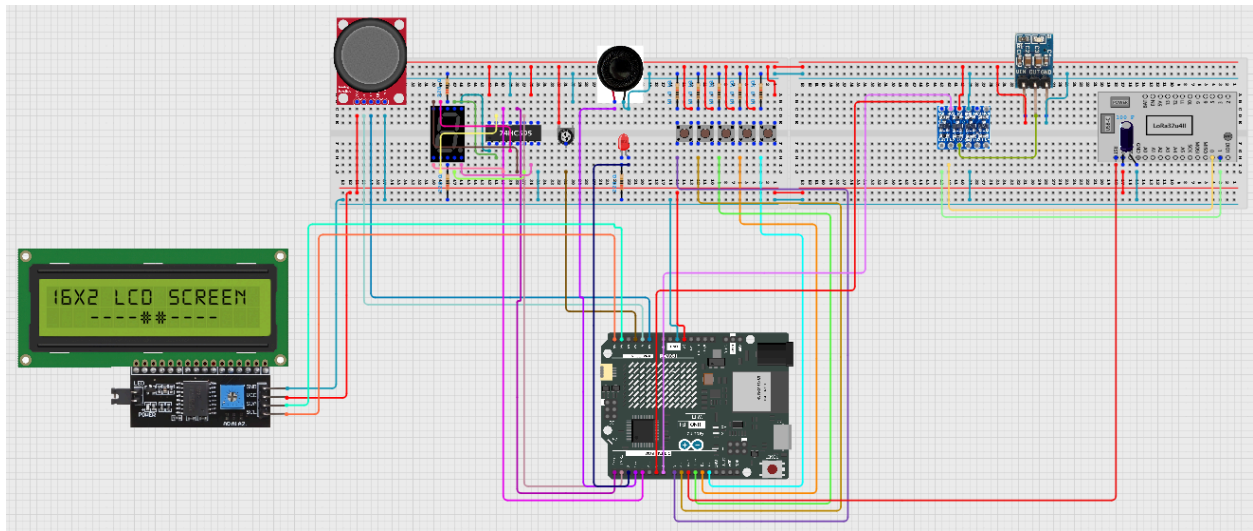
[LoRa module \(amazon\)](#)

https://docs.sunfounder.com/projects/elite-explorer-kit/en/latest/basic_projects/29_basic_74hc595.html

<https://projecthub.arduino.cc/ratack0/seven-segment-display-with-a-74hc595-shift-register-2592a3#section0>

<https://app.cirkitdesigner.com/project/d8403130-ffd6-4635-a0d8-92fb653e2ebf>

Wiring Diagram:



Code:

```
//Names, NetIDs, emails:
//Jackson LeJeune, jleje2, jleje2@uic.edu
//Dean Tubongbanua, dtubon2, dtubon2@uic.edu
//Alec Sanders, asand41, asand41@uic.edu
//Bryan Dominguez, bdomi2, bdomi2@uic.edu

//Group Number: 22
//Project Name: Arduino Telegraph
//Project Abstract: Using Arduino boards, LCD screens, LoRa radio transceivers,
and a few //buttons, we will create a system that, given input of dots and
dashes, will send an encoded //message to other Arduino boards, using radio
signals, that will decode the message and display it //on an LCD screen. Users
will select specific “channels” to broadcast to using a joystick, and any //other
Arduinos on that channel will receive any message broadcasted on it.


//Alec: Version 4/24:
//NOTE: TESTED
/*
-----Change Log-----

//Changelog 4/24:
- potentiometer now correctly controls the volume of the buzzer


//Changelog 4/22 V3-V7: (see below)

Various fixes to state switching
- exiting draft state when pressing clear with empty input buffer
- preventing dot/dash input during playback state
```

- When the send button is pressed during playback when there are no more messages in queue, the LCD displays "END OF MESSAGES. PRESS CLEAR"

message wrapping/validation to protect from UART/RF noise from passing as messages

- all non-command messages now prefixed with '<' and suffixed with '>'

decomposed debounce logic into a helper function (more compact)

```
//-----//
```

```
//Changelog 4/22 V2: (see below)
```

Implemented Huffman encoding/decoding

- I wanted to implement a dynamic huffman encoding algorithm but there would have to be a freq map key prepended to the message that would probably add so much overhead that it would completely defeat the purpose, especially for short messages. This code instead implements a fixed frequency map based on the average frequency of english characters.

-Messages flagged as commands will not be encoded. I want to avoid the need for a decoding algorithm in the LoRa firmware, and the point of encoding is to save time on air. There is no point in encoding a message that will not be sent over RF.

-I tested this by modifying the LoRa firmware to echo any message sent to it back to the arduino as if it were an incoming message. It seems to work, but further testing in an actual LoRa<->LoRa scenario is needed.

//-----///

//Changelog 4/22: Corrected logic to handle 7-seg updates to current channel

//-----///

//Changelog 4/9: (see below)

Added features:

support for typed commands (prepended by a '!' followed by a 2 character command ID)

Joystick input sends channel switching command to LoRa

-In the case of a channel change, the command argument is a frequency value in MHz (915.0MHz - 917.0MHz broken into 10 200KHz steps, ex. !CH915.4)
7-Seg display is updated to current channel based on LoRa's response to command confirming the LoRa's current frequency

Organization:

Moved all helper functions above setup to increase readability.

//-----///

//Changelog 4/7: Fixed space adding two spaces after word

//-----///

//Changelog 4/6: (see below)

SOFTWARE CHANGES

1. Added support for LoRa UART communication
2. Added conditional statement to decide between SoftwareSerial and Serial1 depending on board type
 - a) MEGA2560 TX/RX pins are 18(TX)/19(RX). LoRa TX/RX are 1(TX)/0(RX)

3. Added scrolling output for received LoRa messages

a) Input buffer is still 16 chars. I don't know if we want to allow for longer strings, but I think we should. If so, we can implement scrolling for input as well.

-- In any case, we need to adjust input logic to either explicitly set a 16 char cap or handle scrolling input to avoid overwriting memory outside of the array.

4. Adjusted debounce logic

a) extended debounce time from 10-200ms

-- This seems like a lot, but the debounce for me is really finicky and I don't see rapid input for morse chars as very likely

morse input is tedious, most people don't know the alphabet really well, and I personally feel like accurate parsing of button presses

is more important than speed and responsiveness in this case.

b) implemented separate debounce variables for each button

HARDWARE CHANGES

I was having issues getting serial communication to work. It turned out to be a simple coding error but I was momentarily scared I broke the LoRa. this is because when the Arduino

sends messages to the LoRa over UART through its RX pin, it is an unregulated 5V signal. These signals are really brief and unlikely to cause any issues in the short term

(thus, why nothing has broken so far). Still in order to avoid risking damage to the LoRa module, I STRONGLY recommend passing TX/RX through a logic level shifter to step incoming

voltage down to 3.3V. This will also have the added benefit of stepping outgoing signals up to 5V, which is more appropriate for the Arduino.

I also removed the RST pin connection. The LoRa modules have a reset button on board so connecting it to the Arduino is redundant.

```
////////////////////////////////////  
////////////////////////////////////  
////////////////////////////////////
```

-----TODO-----

Implement a "message waiting" feature: when writing a message, what if the LoRa chip stores the most recent message received, indicated by LED

Return to initial state to allow another message to be drafted and sent.

Implement code to support LED and Buzzer output. Synchronize LED output with buzzer tones

Either implement scrolling input or limit input to 16 chars to prevent exceeding the buffer size

(IN PROGRESS) Additional I/O devices:\

- RGB LED

- LEDs

- Potentiometer & Speaker

(IN PROGRESS) State switching (draft, playback, ready)

(IN PROGRESS) Message queue/message

(DONE) Convert string to huffman encoded binary before sending

(DONE) decode huffman encoded string into char array and Morse String upon receipt

(DONE) Compartmentalization/Code Decomposition

(DONE) Handle joystick input and implement channel switching functionality

(DONE) Install LCD on receiver device and test correct display of received messages

(DONE) Add logic level shifter to circuit diagram

-----KNOWN ISSUES-----

Input buffer for LCD display is initialized to 16 chars, but input is not limited possibly leading to overflow and undefined behavior

-----IDEAS-----

Instead of the top line of LCD simply printing "INCOMING" on receipt of message, perhaps scroll Morse string

```
////////////////////////////////////  
////////////////////////////////////  
////////////////////////////////////
```

```
//Header Comment:  
//Arduino Telegraph by Group 22 -- Arduino master code  
//Members: Alec Sanders, Bryan Dominguez, Dean Tubongbanua, Jack LeJeune  
//  
//Overview: Using Arduinos and LoRa radio communication / microcomputers, create  
a multi-channel telegraph system that communicates messages over radio channels  
//  
//  
//User Interaction:  
//Users can write and receive messages on their individual Arduino boards  
//Writing: Users write messages using Morse code with the buttons on the  
breadboard. Buttons are: dot, dash, space/send message, backspace, and receive  
message. The message in progress  
//is displayed on their LCD screen.
```

```
//Dot and dash create characters from a morse table, space advances the
character, and backspace removes the last finished character. Two spaces
separates words, and the send button sends the message on current channel
//Receiving:
//Once a user decides they want to see the most recent message, they press the
receive button, which displays it on their LCD
```

```
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
*/
```

```
//INCLUDES &
DEFINITIONS-----
//Imports LCD library, sets up the LCD screen with according pin numbers
#include <LiquidCrystal.h>
#include <LiquidCrystal_I2C.h>
#define VRX_PIN A0
#define VRY_PIN A1
```

```
//huffman codec, map, vector and queue Libraries (for encoding/decoding/message
queue)
#define HUFFMAN_CODEC_H
#include <Arduino.h>
#include <map>
#include <queue>
#include <vector>
```

```

//LoRa serial setup-----
//uses software serial for R4/R3 and serial1 for MEGA
#if defined(__AVR_ATmega2560__)
#define LoRaSerial Serial1
#else
#include <SoftwareSerial.h>
#define LORA_RX 6
#define LORA_TX 7
SoftwareSerial LoRaSerial(LORA_RX, LORA_TX);
#endif

//LCD setup
LiquidCrystal_I2C lcd(0x27, 16, 2);

//Digital pins for button
input-----

const int dotPin = 8;
const int dashPin = 9;
const int spacePin = 11;
const int clearPin = 12;
const int sendPin = 13;

//State/message waiting
declarations-----
-----
std::queue<String> messageQueue;
bool draftState = false;
bool playBackState = false;
bool readyState = true;

```

```
bool messageWaiting = false;
```

```
// Scrolling message
```

```
variables-----
```

```
char scrolledMessage[128] = " "; //Buffer to hold message
```

```
int scrollPosition = 0;
```

```
unsigned long lastScrollTime = 0;
```

```
const int scrollDelay = 300;
```

```
int scrolledMessageLength = 0;
```

```
bool newMessageReady = false;
```

```
//OUTPUT
```

```
VARIABLES-----
```

```
-
```

```
const int ledPin = 2;
```

```
const int buzzerPin = 3;
```

```
const int potPin = A2;
```

```
// Button/debounce
```

```
variables-----
```

```
bool dotPressed = false;
```

```
bool dashPressed = false;
```

```
bool spacePressed = false;
```

```
bool clearPressed = false;
```

```
bool sendPressed = false;
```

```
//Debouncing for each button
```

```
//Changed to address inconsistent debounce with fast button presses
unsigned long prevDebounceDot = 0;
unsigned long prevDebounceDash = 0;
unsigned long prevDebounceSpace = 0;
unsigned long prevDebounceClear = 0;
unsigned long prevDebounceSend = 0;
const unsigned long debounceDelay = 200; // extended debounce delay to improve
consistency
```

```
//JOYSTICK
VARIABLES-----
//It is worth noting that due to the position of the joystick in the breadboard,
the y axis is the left and right inputs (positive y = left, and negative y =
right), and "up" is positive x
const int joystickDPin = 7;
int joystickX = 0;
int joystickY = 0;
bool joystickMoved = false;
```

```
//7-Segment display
variables-----
//Pins for 7-segment output
const int latchPin = 0;
const int clockPin = 1;
const int dataPin = 4;
```

```
int leds = 0;
int prevLeds = 0;
int displayChannel = 0;
int lastDisplayChannel = -1;
```

```
float channelMHz = 915.0;
```

```
//Parallel arrays for morse-to-char functionality
```

```
char morseMap[51][7] = { ".-   ", "-... ", "-.-. ", "-..  ", ".    ", "...-.",
"-... ", ".... ", "..   ", ".--- ",
      "-.-  ", ".-.. ", "--   ", "-.   ", "---  ", ".-.-.",
"-...- ", ".-.. ", "...   ", "-     ",
      "-.-  ", "...- ", ".--   ", "-...- ", "-.-.- ", "-....",
"-..... ", ".-----", ".-----", ".-----",
      "..... ", "..... ", "-..... ", "-..... ", "-..... ", "-.....",
"-..... ", ".-----", "-.....", "-..... ",
      "-.....", "-.....", "-..... ", "-.....", "-.....",
"-.....", "-...- ", "-...- ", "-...-.", "-...-.", "-...-.", "-...-." };
char charMap[51] = { 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J',
                    'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T',
                    'U', 'V', 'W', 'X', 'Y', 'Z', '0', '1', '2', '3',
                    '4', '5', '6', '7', '8', '9', '&', '@', ')', '(',
                    ':', ',', '=', '!', '.', '-', '*', '+', '"', '?', '/' };;
```

```
//LCD display strings: top line shows morse input, bottom shows message being
written
```

```
char sendMessageLine[18] = "                \0";
char prevSendMessageLine[18] = "                \0";
```

```
//Holds current morse character
```

```
char morseToCharArray[7] = "        ";
```

```

//Cursor positions
int messageCursorPos = 0;
int morseCodePos = 0;


//Message state
bool messageSent = false;


////////////////////////////////////HELPER
FUNCTIONS////////////////////////////////////


//--LCD AND DISPLAY FUNCTIONS--//


//prepares messages to be scrolled
void prepareScrollMessage(String message) {
    // strip prefix and suffix
    if (message.startsWith("<") && message.endsWith(">")) {
        message = message.substring(1, message.length() - 1);
    }
    message.toCharArray(scrolledMessage, sizeof(scrolledMessage));
    scrolledMessageLength = strlen(scrolledMessage);


    // Pad with spaces to mark end
    strcat(scrolledMessage, "          ");
    scrolledMessageLength = strlen(scrolledMessage);

```



```
scrollPosition = 0;
newMessageReady = true;
```

```
    lcd.clear();
    lcd.setCursor(3, 0); // Top line header
    lcd.print("INCOMING:");
    Serial.print("Incoming plain text: ");
    Serial.println(message);
}
```

```
//helper function to scroll message
void scrollReceivedMessage() {
    if (!newMessageReady) return;
    if (millis() - lastScrollTime >= scrollDelay) {
        lastScrollTime = millis();
        lcd.setCursor(0, 1);
        for (int i = 0; i < 16; i++) {
            int charIndex = (scrollPosition + i) % scrolledMessageLength;
            lcd.print(scrolledMessage[charIndex]);
        }
        scrollPosition++;
        if (scrollPosition >= scrolledMessageLength) {
            scrollPosition = 0;
        }
    }
}
```

```
//helper function to reset LCD
```

```
void resetLCD() {  
    messageCursorPos = 0;  
    morseCodePos = 0;  
    lcd.clear();  
    lcd.setCursor(0, 1);  
    Serial.println("LCD reset.");  
}
```

```
void showReadyPrompt() {  
    newMessageReady = false;  
    lcd.clear();  
    lcd.setCursor(0, 0);  
    Serial.println("In ready state,");  
    lcd.print("Ready for input!");  
}
```

```
//--BUTTON FUNCTIONS--//
```

```
//Handle input from buttons: adds dots/dashes, space, clear, or sends full  
message
```

```
void buttonInput(char c) {  
    if (!draftState && readyState && !playBackState && (c == '.' || c == '-')) {  
        draftState = true;  
        readyState = false;  
        newMessageReady = false; // stop prev scroll  
    }  
    // Block dot/dash input while in playback  
    if (playBackState && (c == '.' || c == '-' || c == ' ')) {
```

```
Serial.println("Drafting input ignored – still in playback mode.");  
return;  
}
```

```
if (messageSent) {  
    for (int i = 0; i < 16; i++) {  
        sendMessageLine[i] = ' '  
    }  
    messageCursorPos = 0;  
    messageSent = false;  
}
```

```
// handle dot  
if (c == '.') {  
    if (morseCodePos < 6) {  
        morseToCharArray[morseCodePos] = '.';  
        morseCodePos++;  
        sendMessageLine[messageCursorPos] = mapMorseCode();  
    } else {  
        sendMessageLine[messageCursorPos] = '#';  
        messageCursorPos++;  
        for (int i = 0; i < 6; i++) { morseToCharArray[i] = ' ' ; }  
        morseCodePos = 0;  
    }  
}
```

```
//handle dash  
else if (c == '-') {  
    if (morseCodePos < 6) {  
        morseToCharArray[morseCodePos] = '-';
```

```

        morseCodePos++;
        sendMessageLine[messageCursorPos] = mapMorseCode();
    } else {
        sendMessageLine[messageCursorPos] = '#';
        for (int i = 0; i < 6; i++) { morseToCharArray[i] = ' '; }
        morseCodePos = 0;
    }
}

```

```

//handle space
else if (c == ' ') {
    if (morseToCharArray[0] == ' ') {
        sendMessageLine[messageCursorPos] = ' ';
        messageCursorPos++;
    } else {
        messageCursorPos++;
        morseCodePos = 0;
        for (int i = 0; i < 6; i++) { morseToCharArray[i] = ' '; }
    }
}

```

```

//handle clear
else if (c == '_') {
    // erase full Morse buffer if currently entering a letter
    if (morseCodePos > 0) {
        morseCodePos = 0;
        for (int i = 0; i < 6; i++) morseToCharArray[i] = ' ';
        if (messageCursorPos < 16) {
            sendMessageLine[messageCursorPos] = ' '; // Clear character preview
        }
    }
}

```

```
// if Morse buffer is empty, remove last entered letter
else if (morseToCharArray[0] == ' ' && messageCursorPos > 0) {
    messageCursorPos--;
    sendMessageLine[messageCursorPos] = ' ';
}
```

```
// check if the entire input is now empty – return to ready state
bool morseEmpty = true;
for (int i = 0; i < 6; i++) {
    if (morseToCharArray[i] != ' ') {
        morseEmpty = false;
        break;
    }
}
```

```
bool messageEmpty = true;
for (int i = 0; i < 16; i++) {
    if (sendMessageLine[i] != ' ') {
        messageEmpty = false;
        break;
    }
}
```

```
//return to ready state if clear is pressed and draft is empty
if (draftState && morseEmpty && messageEmpty) {
    Serial.println("Cleared all input – returning to ready state.");
    draftState = false;
    readyState = true;
}
```

```

    showReadyPrompt();
    newMessageReady = false;
}

// Case 4: Update LCD unless we're back in ready state
if (!readyState) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print(morseToCharArray);
    Serial.print("LCD displaying: ");
    Serial.println(morseToCharArray);
    lcd.setCursor(0, 1);
    lcd.print(sendMessageLine);
    Serial.print("LCD displaying: ");
    Serial.println(sendMessageLine);
}
}

//handle send
else if (c == '+') {
    String plainText = String(sendMessageLine);
    plainText.trim();
    // If it starts with '!', treat it as a command – send as-is
    if (plainText.startsWith("!")) {
        Serial.print("Sending Command: ");
        Serial.println(plainText);
        LoRaSerial.println(plainText);
    } else {
        // Wrap text with < and > as "<message>" to distinguish from UART/RF noise
        String wrapped = "<" + plainText + ">";
        // Encode as Huffman binary string
        String encoded = huffmanEncode(wrapped);
        Serial.print("Outgoing plain text: ");

```

```
    Serial.println(plainText);
    Serial.print("Outgoing binary string: ");
    Serial.println(encoded);
    LoRaSerial.println(encoded);
}
```

```
// Clear buffers and reset
for (int i = 0; i < 6; i++) morseToCharArray[i] = ' ';
for (int i = 0; i < 16; i++) sendMessageLine[i] = ' ';
resetLCD();
messageSent = true;
}
```

```
// LCD redraw, unless in ready state
if (!readyState) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print(morseToCharArray);
    Serial.print("LCD displaying: ");
    Serial.println(morseToCharArray);
    lcd.setCursor(0, 1);
    lcd.print(sendMessageLine);
    Serial.print("LCD displaying: ");
    Serial.println(sendMessageLine);
}
}
```

```
//helper for state dependent send press
void handleSendPress() {
    if (draftState) {
```

```

        buttonInput('+');
        draftState = false;
        readyState = true;
        showReadyPrompt();

    } else if (playBackState) {
        if (!messageQueue.empty()) {
            String encoded = messageQueue.front();
            messageQueue.pop();

            String decoded = huffmanDecode(encoded);
            prepareScrollMessage(decoded);

            if (messageQueue.empty()) {
                messageWaiting = false;
                leds &= 0b01111111; // Turn off DP segment
                updateShiftRegister();
            }
        }
        else {
            // No more messages in queue
            newMessageReady = false;
            lcd.clear();
            lcd.setCursor(2, 0);
            lcd.print("END OF MESSAGES");
            lcd.setCursor(0, 1);
            lcd.print("Press Clear...");
        }
    }
}

```



```

        Serial.println("No more messages. Displayed END OF MESSAGES.");
    }

} else if (messageWaiting && readyState) {
    readyState = false;
    playBackState = true;

    if (!messageQueue.empty()) {
        String encoded = messageQueue.front();
        messageQueue.pop();

        String decoded = huffmanDecode(encoded);
        prepareScrollMessage(decoded);

        if (messageQueue.empty()) {
            messageWaiting = false;
            leds = prevLeds;
            updateShiftRegister();
        }
    }
}
}

```

```

bool debouncedPress(int pin, bool& pressed, unsigned long& lastPressTime) {
    if (digitalRead(pin) == HIGH) {
        if (!pressed && millis() - lastPressTime > debounceDelay) {
            pressed = true;
            lastPressTime = millis();
            return true;
        }
    } else {
        pressed = false;
    }
    return false;
}

```

```

//--7-SEG FUNCTIONS--//

```

```

//Update the seven-segment display with the number functions.

```

```

//implementation based on post on arduino forum:

```

```

https://forum.arduino.cc/t/7-segment-display-with-shift-register-74hc595-and-push-button/582423/5

```

```

void updateSevenSeg(int channel) {

```

```

    // Shift register terminals/segments:

```

```

    // Q0, Q1, Q2, Q3, Q4, Q5, Q6, Q7

```

```

    // D   B   A   E   F   G   C   DP(N/A)

```

```

    const byte segmentMap[10] = {

```

```

        0b01011111, // 0 = A B C D E F

```

```

    0b01000010, // 1 = B C
    0b00101111, // 2 = A B D E G
    0b01100111, // 3 = A B C D G
    0b01110010, // 4 = B C F G
    0b01110101, // 5 = A C D F G
    0b0111101,  // 6 = A C D E F G
    0b01000110, // 7 = A B C
    0b01111111, // 8 = A B C D E F G
    0b01110111  // 9 = A B C D F G
};
if (channel >= 0 && channel <= 9) {
    leds = segmentMap[channel];
    prevLeds = leds;
} else {
    leds = 0;
}
updateShiftRegister();
}

```

//Functions to display numbers on the seven-segment
 //Code provided from here:
<https://projecthub.arduino.cc/ratack0/seven-segment-display-with-a-74hc595-shift-register-2592a3#section0>

```

void updateShiftRegister() {
    digitalWrite(latchPin, LOW);
    // set the latch pin to low so that the 74HC595 can receive data
    shiftOut(dataPin, clockPin, MSBFIRST, leds);
    //write the byte leds to the register
    digitalWrite(latchPin, HIGH);
    //set latch pin back to high
}

```

//--ENCODING/DECODING FUNCTIONS--//

```

//Maps morse code to a character
char mapMorseCode() {
    if (morseToCharArray[0] == ' ') {
        return ' ';
    }
    for (int i = 0; i < 51; i++) {
        if (strcmp(morseToCharArray, morseMap[i]) == 0) {
            return charMap[i];
        }
    }
    return '#';
}

```

```

//Node struct for huffman tree

```

```

struct HuffNode {
    char c;
    unsigned freq;
    HuffNode *left, *right;

    HuffNode(char c, unsigned freq)
        : c(c), freq(freq), left(nullptr), right(nullptr) {}
};

```

```

//Compare for priority queue

```

```

struct Compare {
    bool operator()(HuffNode* l, HuffNode* r) {

```

```

        return l->freq > r->freq;
    }
};

```

//Freq map (based on avg frequency of english characters)

```

std::map<char, unsigned> freqMap = {
    { 'E', 127 }, { 'T', 91 }, { 'A', 82 }, { 'O', 75 }, { 'I', 70 }, { 'N', 67 },
    { 'S', 63 }, { 'H', 61 }, { 'R', 60 }, { 'D', 43 }, { 'L', 40 }, { 'C', 28 }, {
    'U', 28 }, { 'M', 24 }, { 'W', 24 }, { 'F', 22 }, { 'G', 20 }, { 'Y', 20 }, {
    'P', 19 }, { 'B', 15 }, { 'V', 10 }, { 'K', 8 }, { 'X', 2 }, { 'Q', 1 }, { 'J', 1
    }, { 'Z', 1 }, { '0', 5 }, { '1', 5 }, { '2', 5 }, { '3', 5 }, { '4', 5 }, { '5',
    5 }, { '6', 5 }, { '7', 5 }, { '8', 5 }, { '9', 5 }, { '.', 10 }, { ',', 8 }, {
    '?', 6 }, { '!', 6 }, { '-', 6 }, { '=', 4 }, { '(', 2 }, { ')', 2 }, { ':', 2 },
    { '+', 2 }, { '/', 2 }, { '&', 1 }, { '@', 1 }, { '"', 1 }, { ' ', 120 }, { '<',
    100 }, { '>', 100 }
};

```

```

};

```

// Code maps

```

std::map<char, String> codeTable;
std::map<String, char> decodeTable;

```

// build code table

```

void buildCodeTable(HuffNode* root, String code = "") {
    if (!root) return;
    if (!root->left && !root->right) {
        codeTable[root->c] = code;
    }
}

```

```

        decodeTable[code] = root->c;
        return;
    }
    buildCodeTable(root->left, code + "0");
    buildCodeTable(root->right, code + "1");
}

```

//initialize huffman encoding

```

void initHuffman() {
    std::priority_queue<HuffNode*, std::vector<HuffNode*>, Compare> pq;
    for (auto& pair : freqMap)
        pq.push(new HuffNode(pair.first, pair.second));
}

```

```

while (pq.size() > 1) {
    HuffNode* left = pq.top();
    pq.pop();
    HuffNode* right = pq.top();
    pq.pop();
    HuffNode* merged = new HuffNode('\0', left->freq + right->freq);
    merged->left = left;
    merged->right = right;
    pq.push(merged);
}

```

```

    buildCodeTable(pq.top());
}

```

```
// Encode/Decode
String huffmanEncode(String input) {
    String output = "";
    input.toUpperCase();
    for (char c : input) {
        if (codeTable.count(c)) {
            output += codeTable[c];
        }
    }
    return output;
}
```

```
String huffmanDecode(String binary) {
    String decoded = "", current = "";
    for (char b : binary) {
        current += b;
        if (decodeTable.count(current)) {
            decoded += decodeTable[current];
            current = "";
        }
    }
    return decoded;
}
```

```
//////////////////////////////////SETUP//////////////////////////////////
```

```
//Arduino setup: initialize LCD, buttons, and LoRa serial
void setup() {
    Serial.begin(9600);
    //initialize LoRa for UART communication
}
```

```
LoRaSerial.begin(9600);
```

```
//initialize LCD
```

```
//Worth noting that some LCDs require use of lcd.begin() and some use
```

```
//lcd.init(): this is a simple fix when uploading the code, but can't
```

```
//be solved beforehand
```

```
lcd.begin();
```

```
lcd.backlight();
```

```
lcd.print("Ready for input!");
```

```
Serial.println("LCD displaying: Ready for input!");
```

```
readyState = true;
```

```
lcd.setCursor(0, 1);
```

```
lcd.print(" ");
```

```
Serial.print("LCD displaying whitespace on (0, 1).");
```

```
//initialize buttons as inputs
```

```
pinMode(dotPin, INPUT);
```

```
pinMode(dashPin, INPUT);
```

```
pinMode(spacePin, INPUT);
```

```
pinMode(clearPin, INPUT);
```

```
pinMode(sendPin, INPUT);
```

```
pinMode(ledPin, OUTPUT);
```

```
pinMode(buzzerPin, OUTPUT);
```

```
//Set up pins for 7-seg output
```

```
pinMode(latchPin, OUTPUT);
```



```
pinMode(clockPin, OUTPUT);
pinMode(dataPin, OUTPUT);
leds = 0;
updateShiftRegister();
LoRaSerial.println("!CH915.0");
displayChannel = 0;
updateSevenSeg(0);
initHuffman();
}
```

```
//Main loop: scans buttons, handles debounce, and checks LoRa UART input
void loop() {
```

```
    //Debounce dot
    if (debouncedPress(dotPin, dotPressed, prevDebounceDot)) {
        Serial.println("dot");
        buttonInput('.');
    }
```

```
    //Debounce dash
    if (debouncedPress(dashPin, dashPressed, prevDebounceDash)) {
        Serial.println("dash");
        buttonInput('-');
    }
```

```
    //Debounce space
    if (debouncedPress(spacePin, spacePressed, prevDebounceSpace)) {
```

```
    Serial.println("space");
    buttonInput(' ');
}
```

```
//Debounce clear
if (debouncedPress(clearPin, clearPressed, prevDebounceClear)) {
    Serial.println("clear");
    buttonInput('_');
    if (playBackState) {
        playBackState = false;
        readyState = true;
        newMessageReady = false;
        showReadyPrompt();
    }
}
```

```
//Debounce Send
if (debouncedPress(sendPin, sendPressed, prevDebounceSend)) {
    Serial.println("send");
    handleSendPress();
}
```

```
//buzzer logic
if (digitalRead(dotPin) == HIGH || digitalRead(dashPin) == HIGH ||
digitalRead(spacePin) == HIGH || digitalRead(clearPin) == HIGH ||
digitalRead(sendPin) == HIGH) {
```

```

    // play buzzer/flash LED when button is pressed
    int potValue = analogRead(potPin);
    int volume = map(potValue, 0, 1023, 0, 255); // map analog reading to PWM
duty cycle range
    analogWrite(buzzerPin, volume);
    digitalWrite(ledPin, HIGH);
} else {
    // turn off buzzer/LED when button is depressed
    digitalWrite(ledPin, LOW);
    analogWrite(buzzerPin, 0);
}

```

```

//Check for messages from LoRa,
//If not response to command, display message to LCD and scroll
// Persistent buffer to store incoming characters
static String buffer = "";

```

```

// Process all available incoming characters
while (LoRaSerial.available()) {
    char c = LoRaSerial.read();

    if (c == '\n') {
        // We received a complete line
        String received = buffer;
        buffer = ""; // Clear buffer for next message
        received.trim();
    }
}

```

```

    if (received.length() > 0) {
        if (received.startsWith("!")) {
            String cmdID = received.substring(1, 3); // 2-char command ID (e.g.
"CH")
            String cmdArg = received.substring(3); // Argument (e.g. "915.2")

            // Change Channel
            if (cmdID == "CH") {
                channelMHz = cmdArg.toFloat(); // Convert to
float
                displayChannel = (channelMHz - 915.0) / 0.2 + 0.5; // Round to index
                updateSevenSeg(displayChannel);
            }
            // maybe add more commands here
        } else {
            // Otherwise treat it as a message.
            // Validate by looking for encoded '<' prefix and '>' suffix
            String prefix = codeTable['<'];
            String suffix = codeTable['>'];
            if (received.startsWith(prefix) && received.endsWith(suffix)) {
                messageQueue.push(received);
                messageWaiting = true;
                //Update DP (Q7) to indicate message waiting.
                leds |= 0b10000000;
                updateShiftRegister();
            } else {
                Serial.println("Rejected incoming Message: Invalid format or
encoding.");
            }
        }
    }
} else {
    // Append character to ongoing buffer
    buffer += c;
}
}
}

```

```
// Keep scrolling incoming messages unless ready state is triggered
if (!readyState && newMessageReady) {
    scrollReceivedMessage();
}
```

//Joystick input: if the joystick is moved left or right enough, it will change the channel and the seven segment display

```
joystickY = analogRead(VRY_PIN);
if ((abs(joystickY - 516) > 300) && !joystickMoved) {
    joystickMoved = true;
    if (((joystickY - 500) > 0) && displayChannel > 0) {
        LoRaSerial.println("!CH" + String(channelMHz - 0.2, 1));
        Serial.println("Joystick triggered.");
    } else if (((joystickY - 500) < 0) && displayChannel < 9) {
        LoRaSerial.println("!CH" + String(channelMHz + 0.2, 1));
        Serial.println("Joystick triggered.");
    }
}
```

//If the joystick is returned to its neutral position, allow another input to be made

```
if (abs(joystickY - 516) <= 20) {
    joystickMoved = false;
}
```

```
//Needs to be called constantly to keep display on, if called once, doesn't  
store the value
```

```
if (displayChannel != lastDisplayChannel) {  
    updateSevenSeg(displayChannel);  
    lastDisplayChannel = displayChannel;  
}  
}
```